

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №2
по дисциплине «Методы тестирования программного
обеспечения»
«Тестирование ПО методом черного и белого ящика»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Описание методов тестирования	5
2.1 Метод белого ящика	5
2.2 Метод черного ящика	5
3 Программа №1	9
3.1 Описание программы	9
3.2 Тестирование методом белого ящика	10
3.2.1 Критерий покрытия операторов	10
3.2.2 Критерий покрытия решений	11
3.2.3 Критерий покрытия условий	11
3.2.4 Критерий покрытия решений и условий	11
3.2.5 Комбинаторное покрытие	12
3.3 Тестирование методом чёрного ящика	12
3.3.1 Разбиение на классы эквивалентности	12
3.3.2 Анализ граничных значений	13
3.3.3 Причинно-следственная диаграмма	14
4 Программа №2	16
4.1 Описание программы	16
4.2 Тестирование методом белого ящика	18
4.2.1 Критерий покрытия операторов	18
4.2.2 Критерий покрытия решений	18
4.2.3 Критерий покрытия условий	18
4.2.4 Критерий покрытия решений и условий	18
4.2.5 Комбинаторное покрытие	19
4.3 Тестирование методом черного ящика	19
4.3.1 Разбиение на классы эквивалентности	19
4.3.2 Анализ граничных условий	19
4.3.3 Причинно-следственная диаграмма	20
5 Результаты тестирования	22
Заключение	23
Список литературы	24

Введение

В данной лабораторной работе представлено описание использования основных методов модульного тестирования: методов «черного» и «белого» ящика.

Модульное тестирование – это процесс тестирования отдельных блоков, подпрограмм, классов или процедур, образующих крупную программу. Цель модульного тестирования – сравнение функций, реализуемых модулем, со спецификациями, описывающими его функциональные или интерфейсные характеристики.

1 Постановка задачи

При выполнении данной лабораторной работы требуется изучить методы модульного тестирования: метод «белого» ящика и метод «черного» ящика, провести тестирование двух предоставленных другими программистами программ обоими методами.

2 Описание методов тестирования

2.1 Метод белого ящика

В стратегии тестирования методом белого ящика, называемым также тестированием, управляемым логикой программы, разрешается исследовать внутреннюю структуру программы. Исходя из этой стратегии, тестировщик подбирает тестовые данные путем анализа логики программы, которая представляется в виде блок-схемы. Задача данного метода тестирования заключается в том, чтобы охватить тестами все возможные пути выполнения программы. Однако на практике это практически невозможно из-за сложности программ, поэтому применяются различные критерии, которые помогают систематизировать тестирование. Основные из них:

1. Покрытие операторов

Тесты составляются так, чтобы каждый оператор программы исполнялся хотя бы один раз.

2. Покрытие решений

Тесты разрабатываются так, чтобы каждая ветвь условного оператора исполнялась хотя бы один раз.

3. Покрытие условий

Каждое условие внутри условного оператора должно принимать все возможные значения хотя бы один раз.

4. Покрытие решений и условий

В данном подходе тесты составляются так, чтобы каждая ветвь условного оператора исполнялась хотя бы один раз и чтобы каждое из условий внутри условных операторов принимало все возможные значения хотя бы один раз.

5. Комбинаторное покрытие

Тесты разрабатываются так, чтобы каждая возможная комбинация значений всех условий была проверена хотя бы один раз.

2.2 Метод черного ящика

Тестирование методом черного ящика также называют тестированием, управляемым данными, или тестированием, управляемым входом и выходом. В соответствии с этим методом программа рассматривается как “черный ящик”, внутреннее поведение и структура которого не имеют никакого значения. Вместо этого все внимание фокусируется на выяснении обстоятельств, при которых поведение программы не соответствует спецификации.

При таком подходе тестовые данные выбираются исключительно на основе спецификаций требований (без привлечения каких-либо знаний о внутренней структуре программы).

Чтобы в рамках данного метода обнаружить все ошибки в программе, необходимо выполнить так называемое исчерпывающее входное тестирование, т.е. перебрать все возможные комбинации входных данных. Поскольку зачастую проверить все возможные комбинации входных данных невозможно, существует несколько подходов, которые помогают охватить как можно больше сценариев. Основными из них являются:

1. Классы эквивалентности

Определение классов эквивалентности сводится к последовательному рассмотрению каждого из входных условий и разбиению его на две или несколько групп. Эти группы делятся на:

- Допустимые классы эквивалентности, представляющие допустимые входные данные программы;
- Недопустимые классы эквивалентности, представляющие все остальные возможные состояния условий (т.е. недопустимые входные значения).

После определения классов составляются:

- Тесты, которые проверяют как можно больше различных допустимых классов;
- Тесты, каждый из которых проверяет один недопустимый класс.

2. Граничные значения

Для каждого входного значения определяются крайние значения диапазонов допустимых и недопустимых значений. Далее составляются тесты:

- Проверяющие корректность работы программы на границах допустимых значений;
- Проверяющие поведение программы на недопустимых граничных значениях.

3. Причинно-следственная диаграмма

Метод причинно-следственных диаграмм помогает систематически выбирать высокорезультативные тесты. Причинно-следственная диаграмма представляет собой формальный язык, на который транслируется спецификация, написанная на естественном языке. Фактически такая диаграмма является аналогом цифровой логической схемы, использующей простую нотацию. Для построения требуется понимание логических операторов (AND, OR, NOT).

Процесс построения тестов:

- (а) Разбить спецификацию: разделить сложную спецификацию на более мелкие, управляемые части.
- (b) Определить причины и следствия: причины – это входные условия или классы эквивалентности, а следствия – это выходные условия или преобразования системы.
- (с) Построить причинно-следственную диаграмму: создать булев граф, связывающий причины и следствия, отражая семантическое содержание спецификации.
- (d) Добавить ограничения: задать ограничения на комбинации причин и следствий, которые невозможны.
- (е) Преобразовать в таблицу решений: для преобразования графа в таблицу решений необходимо выполнить следующие действия:
 - i. Выбрать следствие, которое должно находиться в состоянии 1 (присутствует).
 - ii. Продвигаясь вдоль линий диаграммы, ведущих к данному узлу, в обратном направлении, найти все комбинации причин (подчиненных ограничениям), которые устанавливают данное следствие в 1.
 - iii. Создать столбец в таблице решений для каждой комбинации причин.
 - iv. Определить для каждой комбинации состояния всех других следствий и поместить их в соответствующий столбец таблицы.

Правила преобразования:

- Если путь обратной трассировки проходит через узел типа *or*, выход которого должен принимать значение 1, то не следует одновременно устанавливать в 1 более одного входа в этот узел. Это ограничение носит название повышение чувствительности пути (*path sensitizing*). Цель данного правила — избежать пропуска некоторых ошибок из-за маскирования одних причин другими.
- Если путь обратной трассировки проходит через узел типа *and*, выход которого должен принимать значение 0, то, конечно же, должны быть перечислены все комбинации входов, приводящие к установке выхода в 0. Однако при исследовании ситуации, в которой один вход равен 0, а один или несколько других входов равны 1, перечислять все условия, при которых другие входы находятся в состоянии 1, не обязательно.
- Если путь обратной трассировки проходит через узел типа *and*, выход которого должен принимать значение 0, то необходимо указать лишь одно условие, при котором все входы являются нулями. (Если узел *and* находится в середине графа и его входы исходят из других

промежуточных узлов, то может существовать чрезвычайно много ситуаций, при которых все его входы принимают значения 0.)

- (f) Преобразовать в тесты: преобразовать столбцы таблицы в конкретные тесты.

Базовая нотация причинно-следственных диаграмм представлена на Рис. 1. Каждый узел диаграммы может находиться в двух состояниях: 0 или 1, где 0 представляет состояние "отсутствует а 1 – "присутствует".

Функция тождество устанавливает следующее: если a равно 1, то и b равно 1; в противном случае b равно 0.

Функция `not` устанавливает следующее: если a равно 1, то b равно 0; в противном случае b равно 1.

Функция `or` устанавливает следующее: если a , или b , или c равно 1, то d равно 1; в противном случае d равно 0.

Функция `and` устанавливает следующее: если и a , и b равно 1, то c равно 1; в противном случае c равно 0.

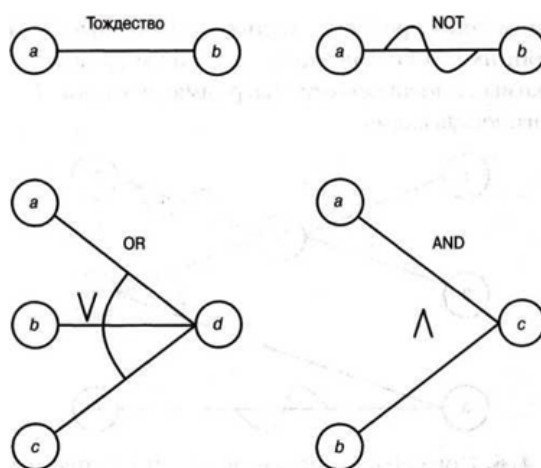


Рис. 1. Базовая нотация, используемая в причинно-следственных диаграммах

3 Программа №1

3.1 Описание программы

Автор: Цыбукова Софья

Название программы: Анализ ориентированного графа методом Шимбелла

Входные данные:

- n – количество вершин графа,
- матрица весов графа $n \times n$,
- K – длина пути,
- r – минимальный или максимальный вес пути для алгоритма Шимбелла.

Требуется: по заданной матрице весов ориентированного графа с помощью алгоритма Шимбелла подсчитать и вывести матрицу, где для каждой пары вершин указан минимальный или максимальный (в зависимости от выбора пользователя) вес пути, состоящего ровно из K рёбер.

Ограничения:

1. $2 \leq n \leq 100$, n – целое число.
2. $1 \leq K$, K – целое число.
3. Элементы матрицы смежности – целые числа от 0 до 9.
4. r может принимать либо значение «min», либо значение «max».
5. При некорректном вводе программа повторяет запрос.

Спецификация программы

Таблица 1. Спецификация программы

Входные данные	Ожидаемый результат	Статус	Комментарий
$n=5$, $K=2$, $r=\text{«min»}$, корректная матрица	Матрица мин. весов путей длины 2	Корректно	Основной сценарий
$n=2$, $K=1$, $r=\text{«min»}$, корректная матрица	Матрица мин. весов путей длины 1	Корректно	Минимальное допустимое n и K
$n=100$, $K=1$, $r=\text{«max»}$, корректная матрица	Матрица макс. весов путей длины 1	Корректно	Максимальное допустимое n
$n=1$	Повтор запроса ввода	Ошибка ввода	Выход за нижнюю границу n
$n=150$	Повтор запроса ввода	Ошибка ввода	Выход за верхнюю границу n
$n=\text{«abc»}$	Повтор запроса ввода	Ошибка ввода	Нечисловой ввод n
$n=5$, $K=0$, корректная матрица	Сообщение об ошибке, повтор ввода	Ошибка ввода	Длина пути должна быть ≥ 1
$n=5$, $K=\text{«два»}$, корректная матрица	Сообщение об ошибке, повтор ввода	Ошибка ввода	Нечисловой K
$n=5$, $K=2$, $r=\text{«other»}$	Повтор запроса режима	Ошибка ввода	Недопустимый режим

Блок-схема алгоритма

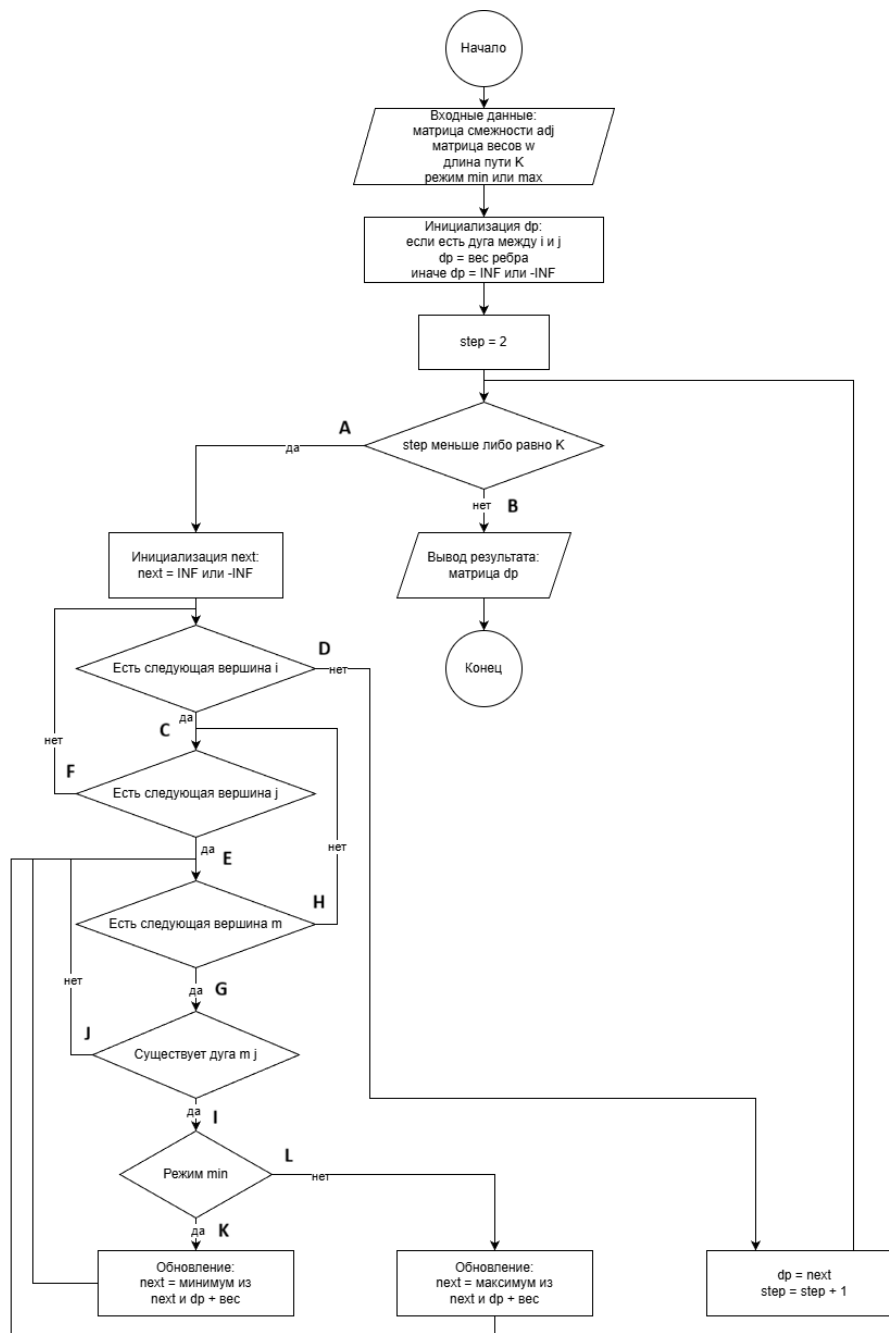


Рис. 2. Блок-схема алгоритма

3.2 Тестирование методом белого ящика

3.2.1 Критерий покрытия операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Критерий является необходимым, но недостаточным условием для полного тестирования по методу белого ящика.

Таблица 2. Тесты покрытия операторов

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=5, матрица, r=«min», K=2	Матрица мин. путей	Матрица мин. путей	A-C-E-J-I-K	T; T; T; T; T; T	Пройден
2	n=5, матрица, r=«min», K=0	Сообщение об ошибке	Сообщение об ошибке	B	F	Пройден
3	n=5, матрица, r=«max», K=2	Матрица макс. путей	Матрица макс. путей	A-C-E-J-I-L	T; T; T; T; T; F	Пройден

3.2.2 Критерий покрытия решений

В соответствии с этим критерием необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное значение, так и ложное. Таким образом, к тестам, составленным для метода покрытия операторов, нужно добавить проверки всех возможных путей выполнения.

Таблица 3. Тесты покрытия решений

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=2, матрица, r=«min», K=1	Матрица путей длины 1	Матрица путей длины 1	A-C-D	T; T; F	Пройден
2	n=2, матрица, r=«min», K=2	Матрица мин. путей	Матрица мин. путей	A-C-E-F	T; T; T; F	Пройден
3	n=3, матрица, r=«min», K=2	Матрица мин. путей	Матрица мин. путей	A-C-E-J-G	T; T; T; T; F	Пройден

3.2.3 Критерий покрытия условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из двух возможных логических значений, true и false, по крайней мере один раз.

Таблица 4. Тесты покрытия условий

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=3, матрица, r=«min», K=«abc»	Сообщение об ошибке, повтор ввода	Сообщение об ошибке, повтор ввода	B	F	Пройден
2	n=3, матрица, r=«min», K=-3	Сообщение об ошибке, повтор ввода	Сообщение об ошибке, повтор ввода	B	F	Пройден

3.2.4 Критерий покрытия решений и условий

Согласно этому критерию тесты необходимо составить таким образом, чтобы каждое условие проверялось хотя бы один раз, каждое решение выполнялось не менее одного раза, и каждый оператор срабатывал хотя бы один

раз. Совокупность всех этих тестов обеспечивает полное покрытие условий и решений. Таким образом, все уже составленные тесты относятся к данному методу.

3.2.5 Комбинаторное покрытие

Согласно критерию комбинаторного покрытия тесты должны охватывать каждую возможную комбинацию значений всех элементарных условий хотя бы один раз. Для данной программы ключевые условия: корректность K (целое, ≥ 1), выбор режима (min/max), наличие дуги между вершинами. Комбинаторный перебор всех сочетаний этих условий не порождает новых тестов сверх уже составленных: каждая значимая комбинация уже покрыта тестами из предыдущих критериев.

Вывод по методу белого ящика (Программа 1). В ходе тестирования методом белого ящика был составлен набор тестов по пяти критериям: покрытия операторов, решений, условий, решений и условий и комбинаторному. В силу недостатка опыта мне не удалось найти ошибки в программе.

3.3 Тестирование методом чёрного ящика

3.3.1 Разбиение на классы эквивалентности

Ниже представлено разбиение на классы эквивалентности, а также тесты для этих классов.

Таблица 5. Классы эквивалентности

Входные дан- ные	Допустимые классы	Недопустимые классы
n	$2 \leq n \leq 100$, n — целое число (A)	$n < 2$, n — целое число (C)
		$n > 100$, n — целое число (D)
		n не целое число (E)
K	$K \geq 1$, K — целое число (B)	$K < 1$, K — целое число (F)
		K не целое число (G)
r	«min» или «max» (H)	другой ввод (I)
Элементы матри- цы	Целые числа от 0 до 9 (J)	Нецелые или вне диапазона (L)

Таблица 6. Тесты для классов эквивалентности

№	Входные данные	Ожидаемый результат	Фактический результат	Классы	Результат
1	$n=5$, корректная матрица, $r=\text{«min»}$, $K=2$	Матрица мин. путей	Матрица мин. путей	A,B,H,J	Пройден
2	$n=1$	Повтор запроса ввода	Повтор запроса ввода	C	Пройден
3	$n=150$	Повтор запроса ввода	Повтор запроса ввода	D	Пройден
4	$n=\text{«abc»}$	Повтор запроса ввода	Повтор запроса ввода	E	Пройден
5	$n=3.5$	Повтор запроса ввода	—	E	Не пройден
6	$n=5$, корректная матрица, $r=\text{«min»}$, $K=0$	Сообщение об ошибке, повтор ввода	Сообщение об ошибке, повтор ввода	F	Пройден
7	$n=5$, корректная матрица, $r=\text{«min»}$, $K=\text{«abc»}$	Сообщение об ошибке, повтор ввода	Сообщение об ошибке, повтор ввода	G	Пройден
8	$n=5$, корректная матрица, $r=\text{«min»}$, $K=3.5$	Сообщение об ошибке, повтор ввода	—	G	Не пройден
9	$n=5$, матрица, $K=2$, $r=\text{«other»}$	Повтор запроса режима	Повтор запроса режима	I	Пройден
10	$n=3$, матрица с элементом «X»	Повтор запроса ввода	Повтор запроса ввода	L	Пройден

3.3.2 Анализ граничных значений

В программе есть следующие граничные условия для числа вершин n :

- Нижняя граница: $n = 2$;
- Верхняя граница: $n = 100$.

Для длины пути K :

- Нижняя граница: $K = 1$.

Для анализа поведения программы при значениях входных данных близких к граничным необходимо протестировать:

- Значения на самих границах;
- Значения непосредственно за границами;
- Типичные значения внутри диапазона (покрыто классами эквивалентности).

Таблица 7. Тесты граничных условий

№	Входные данные	Ожидаемый результат	Фактический результат	Граничное условие	Результат
1	$n=2$, матрица, $r=\text{«min»}$, $K=1$	Матрица мин. путей	Матрица мин. путей	Мин. допустимое n	Пройден
2	$n=1$, матрица	Повтор запроса ввода	Повтор запроса ввода	Ниже мин. границы n	Пройден
3	$n=100$, матрица, $r=\text{«min»}$, $K=1$	Матрица мин. путей	Матрица мин. путей	Макс. допустимое n	Пройден
4	$n=101$, матрица	Повтор запроса ввода	Повтор запроса ввода	Выше макс. границы n	Пройден
5	$n=5$, матрица, $r=\text{«min»}$, $K=1$	Матрица путей длины 1	Матрица путей длины 1	Мин. допустимое K	Пройден
6	$n=5$, матрица, $r=\text{«min»}$, $K=0$	Сообщение об ошибке, повтор ввода	Сообщение об ошибке, повтор ввода	Ниже мин. границы K	Пройден

3.3.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введём следующие обозначения.

Причины:

1. n – целое число;
2. $2 \leq n \leq 100$;
3. K – целое число;
4. $K \geq 1$;
5. r принимает значения «min» или «max»;
6. все элементы матрицы – целые числа от 0 до 9.

Следствия:

20 Программа вычисляет и выводит матрицу Шимбелла;

21 Программа выводит сообщение об ошибке и повторяет запрос.

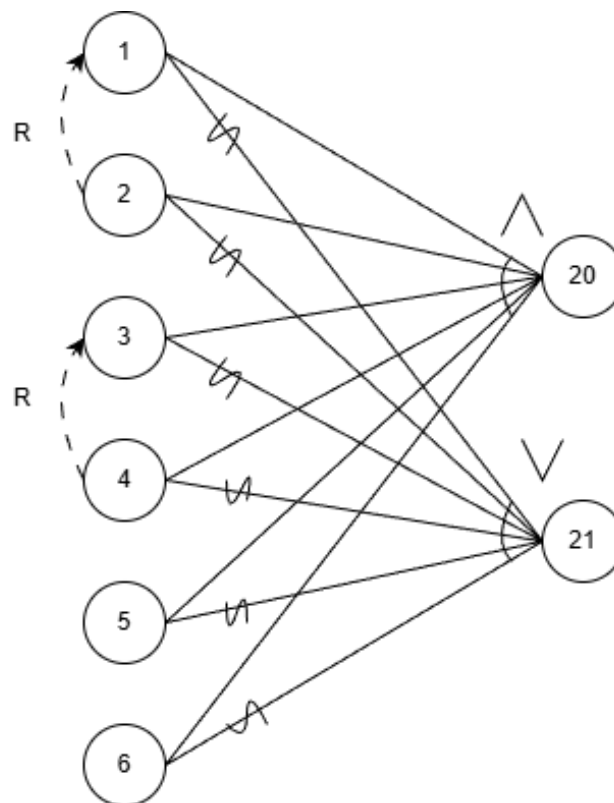


Рис. 3. Причинно-следственная диаграмма

Таблица 8. Таблица решений

№	1	2	3	4	5	6	7	8	9	10	11
1	1	0	1	1	1	1	1	0	0	1	1
2	1	0	0	1	1	1	1	0	0	0	1
3	1	1	1	0	1	1	1	1	1	0	1
4	1	1	1	0	0	1	1	1	1	0	0
5	1	1	1	1	1	0	1	1	1	1	0
6	1	1	1	1	1	1	0	1	1	1	0
20	1	0	0	0	0	0	0	0	0	0	0
21	0	1	1	1	1	1	1	1	1	1	1

Таблица 9. Тесты

№	Входные данные	Ожидаемый результат	Фактический результат	Столбец	Результат
1	n=5, матрица, r=«min», K=2	Матрица Шимбелла	Матрица Шимбелла	1	Пройден
2	n=«abc», прочие кор.	Сообщение об ошибке	Сообщение об ошибке	2	Пройден
3	n=150, прочие кор.	Сообщение об ошибке	Сообщение об ошибке	3	Пройден
4	n=5, K=«abc», прочие кор.	Сообщение об ошибке	Сообщение об ошибке	4	Пройден
5	n=5, K=0, прочие кор.	Сообщение об ошибке	Сообщение об ошибке	5	Пройден
6	n=5, матрица, K=2, r=«other»	Сообщение об ошибке	Сообщение об ошибке	6	Пройден
7	n=5, матрица с «x», min, K=2	Сообщение об ошибке	Сообщение об ошибке	7	Пройден
8	n=«abc», K=«abc», прочие кор.	Сообщение об ошибке	Сообщение об ошибке	8	Пройден
9	n=«abc», K=0, прочие кор.	Сообщение об ошибке	Сообщение об ошибке	9	Пройден
10	n=5, K=0, r=«other»	Сообщение об ошибке	Сообщение об ошибке	10	Пройден
11	n=5, матрица, K=«abc», r=«other»	Сообщение об ошибке	Сообщение об ошибке	11	Пройден

Вывод по методу чёрного ящика (Программа 1). В ходе тестирования методом чёрного ящика были применены три техники: разбиение на классы эквивалентности, анализ граничных значений и составление причинно-следственной диаграммы. В следствие составления тестов выявлен дефект: программа не обрабатывает нецелочисленные значения переменных n и K .

4 Программа №2

4.1 Описание программы

Автор: Васюк Марина

Название программы: алгоритм быстрой сортировки массива

Входные данные:

n – размер массива,

a – массив целых чисел длины n .

Требуется: отсортировать элементы массива a по возрастанию с помощью алгоритма быстрой сортировки и вывести отсортированный массив.

Ограничения:

1. n – целое число: $1 \leq n \leq 10000$

2. Каждый элемент $a[i]$ – целое число (тип `int`)

Спецификация программы

Таблица 10. Спецификация

Входные данные	Ожидаемый результат	Реакция программы
n – целое число, $1 \leq n \leq 10000$; далее вводится ровно n целых чисел, формирующих массив a .	Вывести элементы массива a в порядке возрастания, через пробел, в одной строке.	При корректном n и корректном вводе n целых чисел вызывается функция быстрой сортировки, затем отсортированный массив выводится на экран, после чего программа завершает работу.
$n = 0$	Вывести сообщение об ошибке "Error: incorrect n".	При обнаружении некорректного значения n программа выводит строку "Error: incorrect n" и немедленно завершает работу, не запрашивая и не обрабатывая массив.
$n = 10001$	Вывести сообщение об ошибке "Error: incorrect n".	При обнаружении некорректного значения n программа выводит строку "Error: incorrect n" и немедленно завершает работу, не запрашивая и не обрабатывая массив.
$n = \text{«abc»}$	Вывести сообщение об ошибке "Error: incorrect n".	При обнаружении некорректного значения n программа выводит строку "Error: incorrect n" и немедленно завершает работу, не запрашивая и не обрабатывая массив.
$n = 2.5$	Вывести сообщение об ошибке "Error: incorrect n".	При обнаружении некорректного значения n программа выводит строку "Error: incorrect n" и немедленно завершает работу, не запрашивая и не обрабатывая массив.

Блок-схема алгоритма

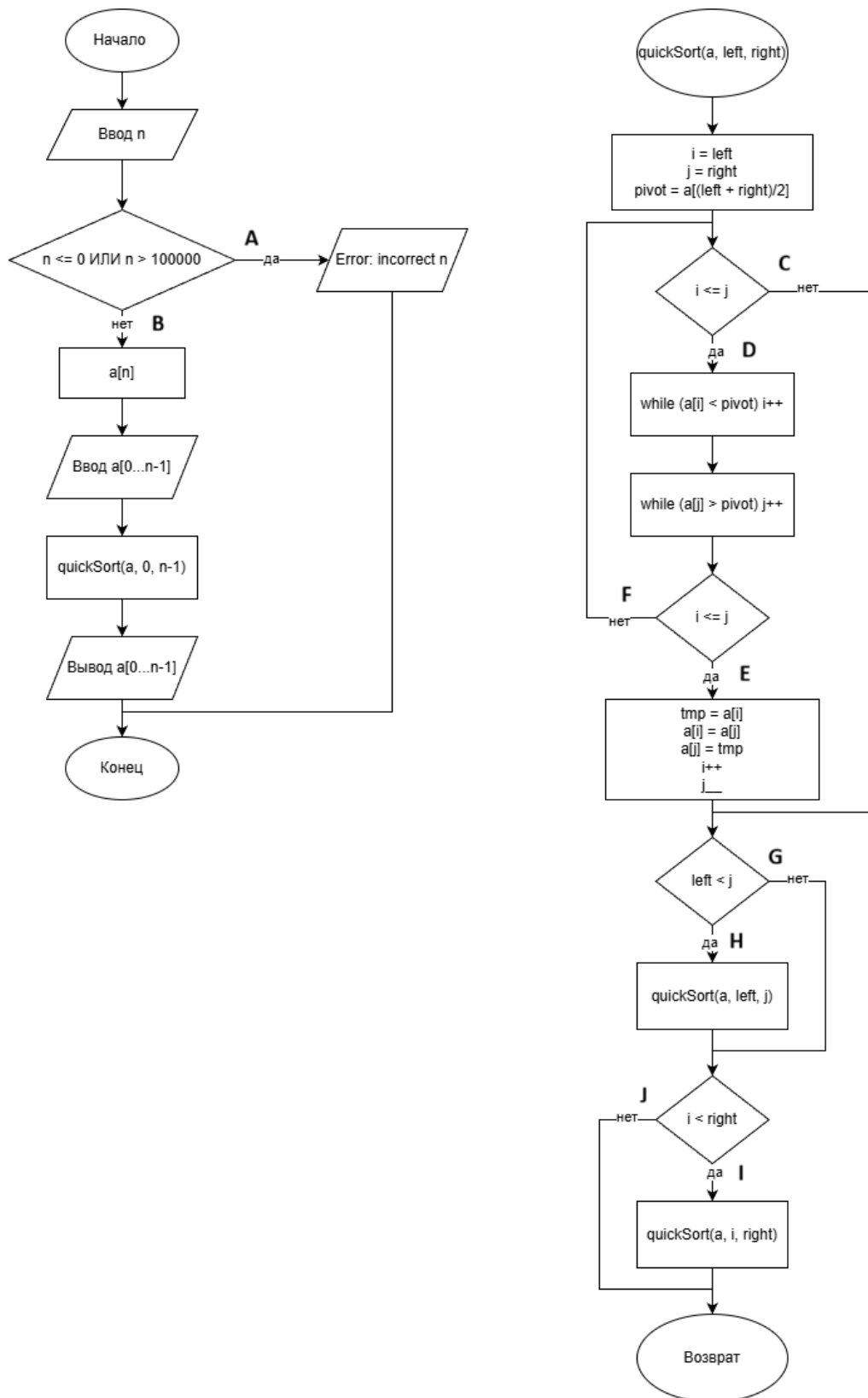


Рис. 4. Блок-схема алгоритма

4.2 Тестирование методом белого ящика

4.2.1 Критерий покрытия операторов

Таблица 11. Тесты покрытия операторов

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=0	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	A	T	Пройден
2	n = 5, a = [3,1,2,5,4]	Отсортированный массив: 1 2 3 4 5	Отсортированный массив: 1 2 3 4 5	B-C-D-E-F-G-H-I-J	F; T; T; F; T; F; F; T; T	Пройден

4.2.2 Критерий покрытия решений

В дополнение к тестам из критерия покрытия операторов были составлены следующие тесты:

Таблица 12. Тесты покрытия решений

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=2, a=[1, 2]	Отсортированный массив: 1 2	Отсортированный массив: 1 2	B-C-D-E-G-J	F; F; T; T; T; T	Пройден
2	n=3, a=[3,2,1]	Отсортированный массив: 1 2 3 4 5	Отсортированный массив: 1 2 3 4 5	B-C-D-F-G-H-J-I	T; F; T; F; F; T; F; T	Пройден
3	n=10001	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	A	T	Пройден

4.2.3 Критерий покрытия условий

Для покрытия всех возможных значений элементарных условий добавим к имеющимся тестам еще несколько:

Таблица 13. Тесты покрытия условий

№	Входные данные	Ожидаемый результат	Фактический результат	Путь	Условия	Результат
1	n=1, a=[5]	Отсортированный массив: 5	Отсортированный массив: 5	B-C-G-J	F; F; F; F	Пройден
2	n=3, a=[2,2,2]	Отсортированный массив: 2 2 2	Отсортированный массив: 2 2 2	B-C-D-F-G-J	F; F; T; F; F; F	Пройден
3	n=4, a=[1, 4, 2, 3]	Отсортированный массив: 1 2 3 4	Отсортированный массив: 1 2 3 4	B-C-D-E-F-G-H-I-J	F; F; T; T; F; F; T; T; F	Пройден
4	n=-1	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	A	T	Пройден

4.2.4 Критерий покрытия решений и условий

Согласно этому критерию тесты необходимо составить таким образом, чтобы каждое условие проверялось хотя бы один раз, каждое решение выполнялось не менее одного раза, и каждый оператор срабатывал хотя бы один раз. Совокупность всех этих тестов обеспечивает полное покрытие условий и

решений. Таким образом, все уже составленные тесты относятся к данному методу.

4.2.5 Комбинаторное покрытие

Согласно критерию комбинаторного покрытия тесты должны охватывать каждую возможную комбинацию значений всех элементарных условий в точках ветвления хотя бы один раз. Полный комбинаторный перебор элементарных условий не порождает тестов, не покрытых предыдущими критериями: все значимые сочетания уже рассмотрены в тестах покрытия операторов, решений и условий.

Вывод по методу белого ящика (Программа 2). Тестирование методом белого ящика охватило все узлы и ветви блок-схемы алгоритма быстрой сортировки по четырём критериям покрытия. В силу недостатка опыта в тестировании мне не удалось найти ошибки в программе.

4.3 Тестирование методом черного ящика

4.3.1 Разбиение на классы эквивалентности

Ниже представлено разбиение на классы эквивалентности, а также тесты для этих классов.

Таблица 14. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
n	$1 \leq n \leq 10000$, n – целое число (A)	$n < 1$, n – целое число (C)
		$n > 10000$, n – целое число (D)
		n не целое число (E)
a[i]	a[i] – целое число (тип int) (B)	a[i] не целое число (тип int) (F)

Таблица 15. Тесты для классов эквивалентности

№	Входные данные	Ожидаемый результат	Фактический результат	Классы эквивалентности	Результат
1	n=5, a=[5, 1, 3, 2, 4]	Массив отсортирован: 1 2 3 4 5	Массив отсортирован: 1 2 3 4 5	A, B	Пройден
2	n=0	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	C	Пройден
3	n=10010	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	D	Пройден
4	n=«abc»	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	E	Пройден
5	n=3, a=[1, «x», 3]	Сообщение об ошибке ввода элемента	–	F	Не пройден
6	n=3, a=[1, 2.5, 3]	Сообщение об ошибке ввода элемента	–	F	Не пройден

4.3.2 Анализ граничных условий

В программе есть следующие граничные условия для размера массива n:

- Нижняя граница: $n = 1$;
- Верхняя граница: $n = 10000$.

Для анализа поведения программы при значениях входных данных близким к граничным необходимо протестировать:

- Значения на самих границах (1, 10000);
- Значения непосредственно за границами (0, 10001);
- Типичные значения внутри диапазона (покрыто классами эквивалентности).

Таблица 16. Тесты граничных условий

№	Входные данные	Ожидаемый результат	Фактический результат	Граничное условие	Результат
1	$n=1, a=[5]$	Массив отсортирован: 5	Массив отсортирован: 5	Мин. допустимое n	Пройден
2	$n=0$	Сообщение об ошибке «Error: incorrect n »	Сообщение об ошибке «Error: incorrect n »	Ниже мин. границы	Пройден
3	$n=10000$	Массив отсортирован по возрастанию	Массив отсортирован по возрастанию	Макс. допустимое n	Пройден
4	$n=10001$	Сообщение об ошибке «Error: incorrect n »	Сообщение об ошибке «Error: incorrect n »	Выше макс. границы	Пройден

4.3.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы необходимо ввести некоторое обозначения.

Причины:

1. n – целое число;
2. $1 \leq n \leq 10000$;
3. все $a[i]$ – целое число.

Следствия:

20 программа выводит отсортированный массив,

21 Программа выводит сообщение об ошибке «Error: incorrect n ».

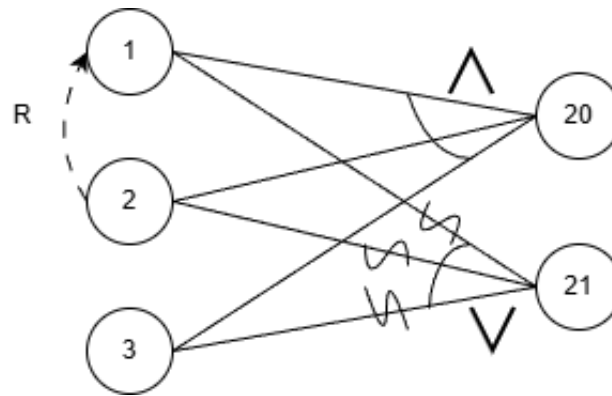


Рис. 5. Причинно-следственная диаграмма

Таблица 17. Таблица решений

№	1	2	3	4
1	1	1	1	0
2	1	1	0	0
3	1	0	0	0
4	1	0	0	0
5	0	0	1	1

Таблица 18. Тесты

№	Входные дан-ные	Ожидаемый ре-зультат	Фактический ре-зультат	Столбец таблицы	Результат
1	n=4, a=[5, 2, 8, 1]	Массив отсортиро-ван: 1, 2, 5, 8	Массив отсортиро-ван: 1, 2, 5, 8	1	Пройден
2	n=3, a=[1.5, 2, 3]	Сообщение об ошибке ввода элемента	–	2	Не пройден
3	n=0	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	3	Пройден
4	n=10001	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	3	Пройден
5	n=«abc»	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	4	Пройден
6	n=2.5	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	4	Пройден
7	n=-5	Сообщение об ошибке «Error: incorrect n»	Сообщение об ошибке «Error: incorrect n»	3	Пройден
8	n=5, a=[1, 2, «x», 4, 5]	Сообщение об ошибке ввода элемента	–	2	Не пройден

Вывод по методу чёрного ящика (Программа 2). В ходе тестирования методом чёрного ящика были применены разбиение на классы эквивалентности, анализ граничных значений и построение причинно-следственной диаграммы. Посредством тестов в программе выявлен дефект: программа не обрабатывает нецелочисленные элементы массива и не выводит сообщение об ошибке при вводе, например, «1.5» или символьных значений.

5 Результаты тестирования

В ходе тестирования программ методом белого ящика было составлено

- 5 теста для метода покрытия операторов (0 не пройдено),
- 6 теста для метода покрытия решений (0 не пройдено),
- 6 теста для метода покрытия условий (0 не пройдено),
- 12 тестов для метода покрытия решений и условий (0 не пройдено),
- 17 тестов для комбинаторного покрытия (0 не пройдено).

В ходе тестирования программ методом черного ящика было составлено

- 16 тестов для метода разбиения на классы эквивалентности (3 не пройдено),
- 10 теста для анализа граничных условий (0 не пройдено),
- 19 тестов по методу причинно-следственной диаграммы (2 не пройдено).

Заключение

В рамках данной лабораторной работы были изучены методы модульного тестирования: метод белого ящика и метод черного ящика. Для двух программ были спроектированы тесты, в которых использовались следующие методы:

- критерий покрытия операторов;
- критерий покрытия решений;
- критерий покрытия условий;
- критерий покрытия решений и условий;
- комбинаторное покрытие;
- разбиение на классы эквивалентности;
- анализ граничных значений;
- причинно-следственная диаграмма.

Для меня эта работа оказалась полезна тем, что разбираясь в спецификациях и блок-схемах других людей, я поняла, на какие моменты в их составлении стоит обращать внимание мне самой. Также я лучше научилась понимать логику программ по блок-схемам.

Список литературы

1. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.